CS-112
Object Oriented Programming

DEPARTMENT OF COMPUTER SCIENCE

COURSE INSTRUCTOR : MS. HABIBA ARSHAD

Today we will talk about!

- Accessors and Mutators (Getter & Setter)
- Class Diagram
- Conversion of Class diagram into Code

Accessor and Mutator Methods

- In Java, we usually make the fields (variables) inside a class as private to protect them. This is called data hiding.
- To access or change these private fields from outside the class, we create special methods:
  - **Getter (Accessor):** A method that gets or returns the value of a field.
  - **Setter (Mutator):** A method that sets or changes the value of a field.

Getter and Setter

Getter methods,

- A getter method starts with the word "get" followed by the variable name, where the first letter of the variable is capitalized.
- It returns the value of the variable.
- Sometimes, a getter can process the data before returning it, so you don’t have to show the raw value. However, a getter should never change or modify the data—it’s only meant to provide access to it.

Getter and Setter

Setter methods,

- A setter method usually starts with the word "set," followed by the variable name with its first letter capitalized.
- It takes a parameter and assigns it to the class's member variable.
- A setter can also include checks to make sure the data is valid (like checking if a number is in a certain range) or change the data before storing it.

Example Code

```
class Employee{
    private int id;
    private String name;
}
public class SetterAndGetter {

    public static void main(String[] args) {
        Employee emp = new Employee();
        emp.id=10;
        emp.name = "MyName";
        System.out.print("Name:"+ name);

    }
}
```

```java
public class SetterAndGetter {

    public static void main(String[] args) {
        Employee emp = new Employee();
        emp.setId( i:10);
        emp.setName( n:"John");
        System.out.println("Id of Employee is :" + emp.getId());
        System.out.println("Name of employee is "+ emp.getName());

    }
}
```

```java
class Employee{
    private int id;
    private String name;

    public void setName(String n){
        name = n;
    }
    public String getName(){
        return name;
    }
    public void setId(int i){
        id = i;
    }
    public int getId(){
        return id;
    }
}
```

Setter and Getter

Class Diagram

Class diagrams are the main building block in object-oriented modeling.

It is a blueprint for creating objects that share similar roles in a system.

They are used to show the different objects in a system, their attributes, their operations and the relationships among them.

The following figure is an example of a simple class:

Simple class diagram with attributes and operations

Explanation

In the example, a class called “loan account” is depicted. Classes in class diagrams are represented by boxes that are partitioned into three:

- The top partition contains the name of the class.
- The middle part contains the class’s attributes.
- The bottom partition shows the possible operations that are associated with the class.

The example shows how a class can encapsulate all the relevant data of a particular object in a very systematic and clear way. A class diagram is a collection of classes similar to the one above.

Class Diagram

It defines the structure and behavior of objects through:

Structural features (attributes) define what objects of the class **"know“** and

- Represent the state of an object of the class
- Describe the static features of a class

Example: Consider a Car class:

Attributes: color, brand, speed

A specific car object might have:

- color = red
- brand = Toyota
- speed = 120 km/h

The values of these attributes represent the current state of the car object.

Class Diagram

Behavioral features (operations) define what objects of the class **"can do"**

- Define the way in which objects may interact
- Operations are methods or functions that an object can perform

Example:

For the same Car class:

Operations: accelerate(), brake(), honk()

These actions define how the car object behaves and interacts in the system.

UML Class Diagram

A **UML class diagram** is a **specific type of class diagram** defined by Unified Modeling Language.

It is a standardized (follow rules) visual language used to model and design software systems by representing classes, objects, relationships, and interactions in Object-Oriented Programming (OOP).

UML helps developers, designers, and stakeholders understand system architecture and behavior.

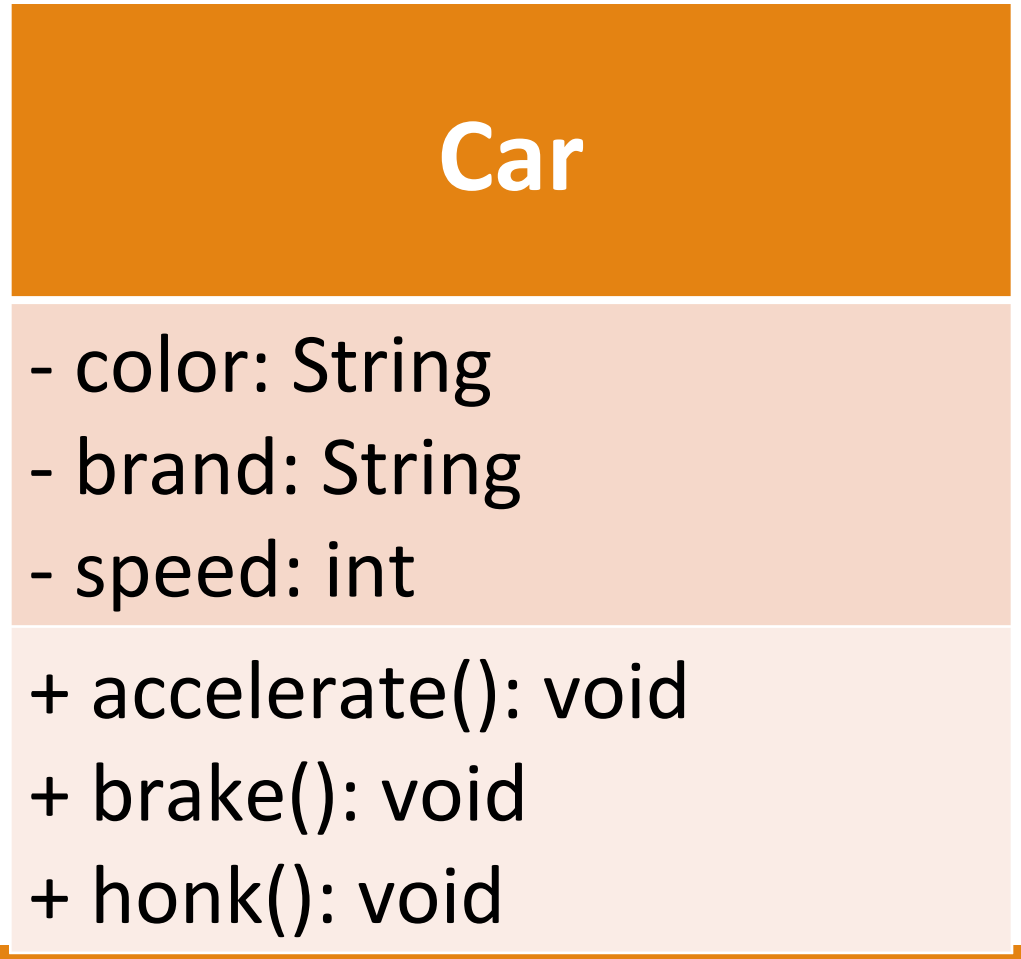
It is a standardized way to create class diagram. All UML class diagrams are class diagrams, but not all class diagrams are UML class diagrams.

UML Class Diagram

It includes well-defined elements like:

- Class name, attributes, operations
- Visibility (+, -, #)
- Relationships (association, inheritance, aggregation, composition)
- Multiplicity (1, *, 0..1, etc.)

UML Class diagram



Class Name

Attributes (Structural Features)

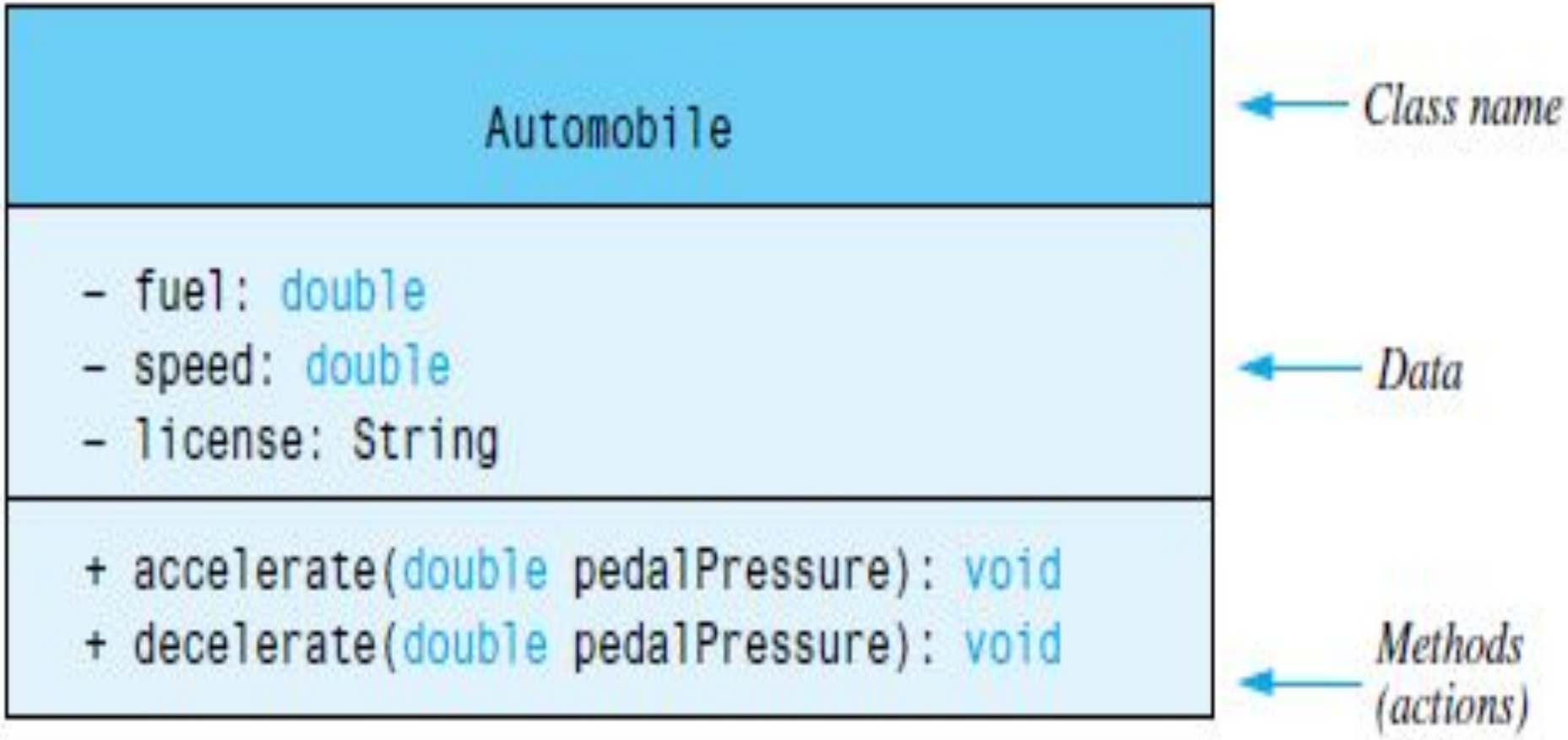
- Means private

+ Means public

: / ~ means protected

Operations (Behavioral Features)

Class Diagram



Class Diagram Relationships

Class Diagram Relationship Type	Notation
Association	
Inheritance	
Realization/ Implementation	
Dependency	
Aggregation	
Composition	

How to Draw a Class Diagram

Here are the steps you need to follow to create a class diagram.

Step 1: Identify the class names

The first step is to identify the primary objects of the system.

Step 2: Distinguish relationships

Next step is to determine how each of the classes or objects are related to one another. Look out for commonalities and abstractions among them; this will help you when grouping them when drawing the class diagram.

Step 3: Create the Structure

First, add the class names and link them with the appropriate connectors. You can add attributes and functions/ methods/ operations later.

https://youtu.be/rtqQeYKHHNg

Example: Scenario

A Customer visits the bank and provides their details such as name, address, and contact_number. The bank system creates a new Customer profile and assigns a unique customer_id. Using the create_account() method, the customer opens a new Account. The account is assigned an account_number, linked to the customer_id, and initialized with an account_type and starting balance.

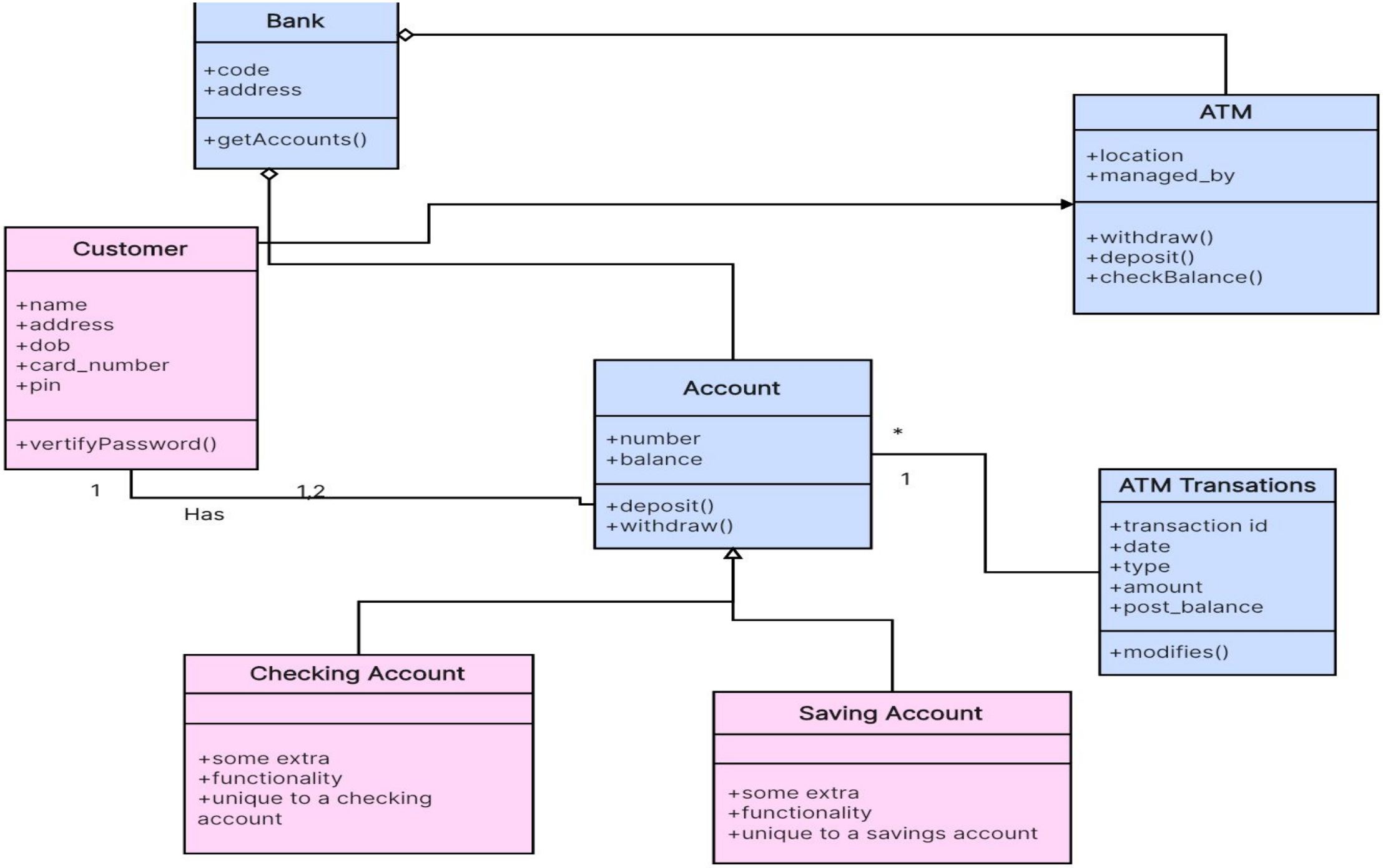
Later, the customer decides to perform a transaction. They deposit money into their Account by calling the credit() method, which updates their balance. The Transaction record is created with a unique transaction_id, showing the account_number, transaction_type as "Credit", the amount, and the date of the transaction.

After maintaining a good balance, the customer applies for a Loan by calling request_loan() from their profile. The Loan class processes this request, generating a loan_id, linking it to the customer_id, and specifying the amount and interest_rate. The system runs the approve_loan() method to finalize the loan approval and later uses calculate_interest() to determine how much interest the customer will owe.

Banking System

Some of the following classes could be used:

- **Customer:** This class could have attributes like customer_id, name, address, and contact_number. Methods could include create_account(), request_loan().
- **Account:** Attributes could include account_number, customer_id, balance, account_type. It could have operations like debit(), credit(), check_balance().
- **Loan:** The loan class could have loan_id, customer_id, amount, interest_rate as attributes, and approve_loan(), calculate_interest() as methods.
- **Transaction:** Its attributes might be transaction id,


Bank
+code
+address
+getAccounts()
ATM
+location
+managed_by
+withdraw()
+deposit()
+checkBalance()
Customer
+name
+address
+dob
+card_number
+pin
+vertifyPassword()
Account
+number
+balance
+deposit()
+withdraw()
*
1
1
1,2
Has
ATM Transations
+transaction id
+date
+type
+amount
+post_balance
+modifies()
Checking Account
+some extra
+functionality
+unique to a checking account
Saving Account
+some extra
+functionality
+unique to a savings account

Scenario Activity

Guest-house management system is an online web based system which allows the guest-house manager to carry out all the activities related to motels. Interactive graphic user interface to manage various bookings and rooms makes this system very convenient and flexible. The guest-house manager is a busy person who have no time to sit and manage all the record manually on a paper. This system allows him to manage the overall system from a single system. Guest-house management system provides booking of room, staff management and other necessary guest house management features. The owner of the guest house can monitor and authorize the tasks handle by the system. Owner can use all the functions performed by the system. The system allows the receptionist to post the available rooms in the system. Customers can view and book all the rooms that are available online. Admin in has the power to approve and disapprove customer booking request. Other services provided by the guest house can also be viewed by the customer and can book them too. Hence the system is useful both for the customers and manager to portable manage the guest house activities. To preserve the security of guest house system the customer must be given a unique login id to access the portal where he can enter his information (name, address, email) and system in return system display the availability of the rooms.

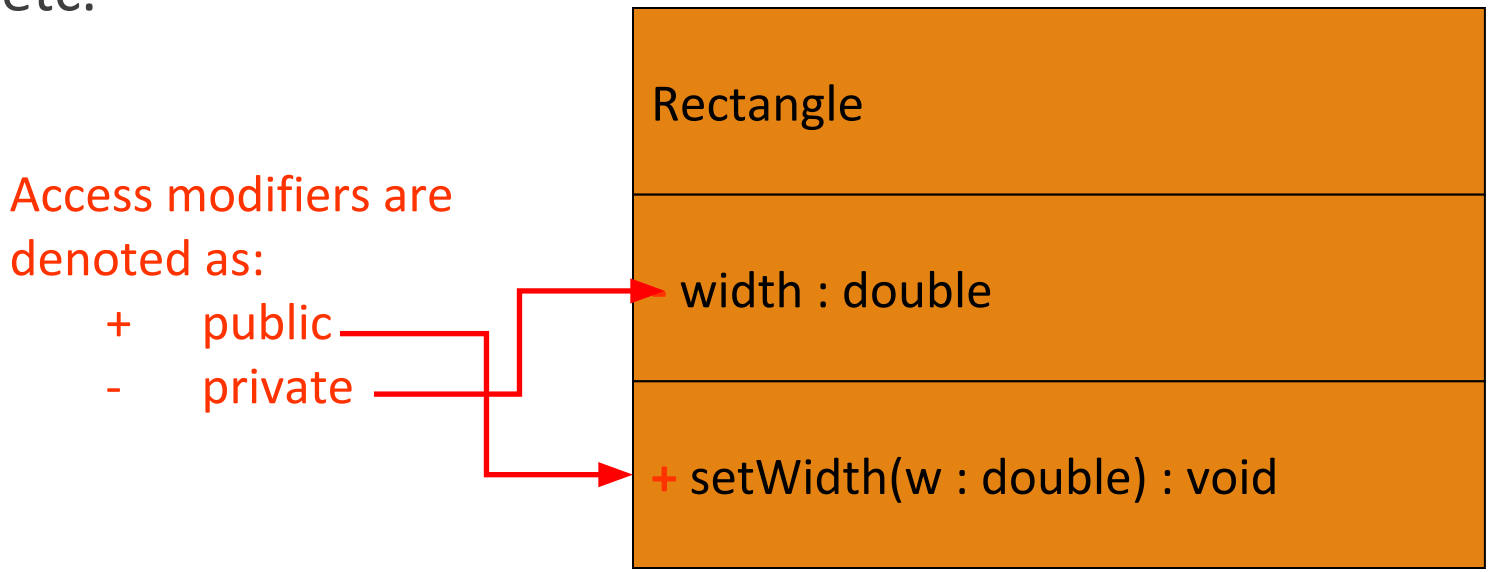
Activity

From stated scenario identify objects, what attributes and operations would be included, their relationships. Also draw a class diagram.

Converting the Class Diagram to Code

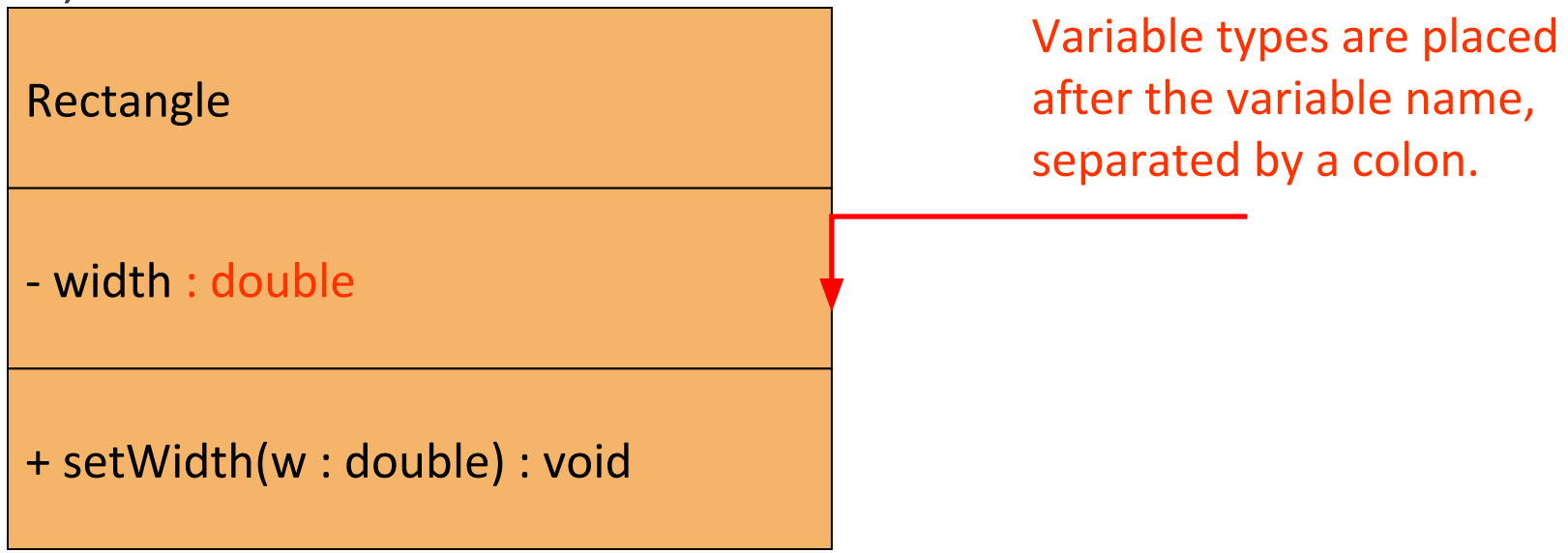
UML Data Type and Parameter Notation

- UML diagrams are language independent.
- UML diagrams use an independent notation to show return types, access modifiers, etc.



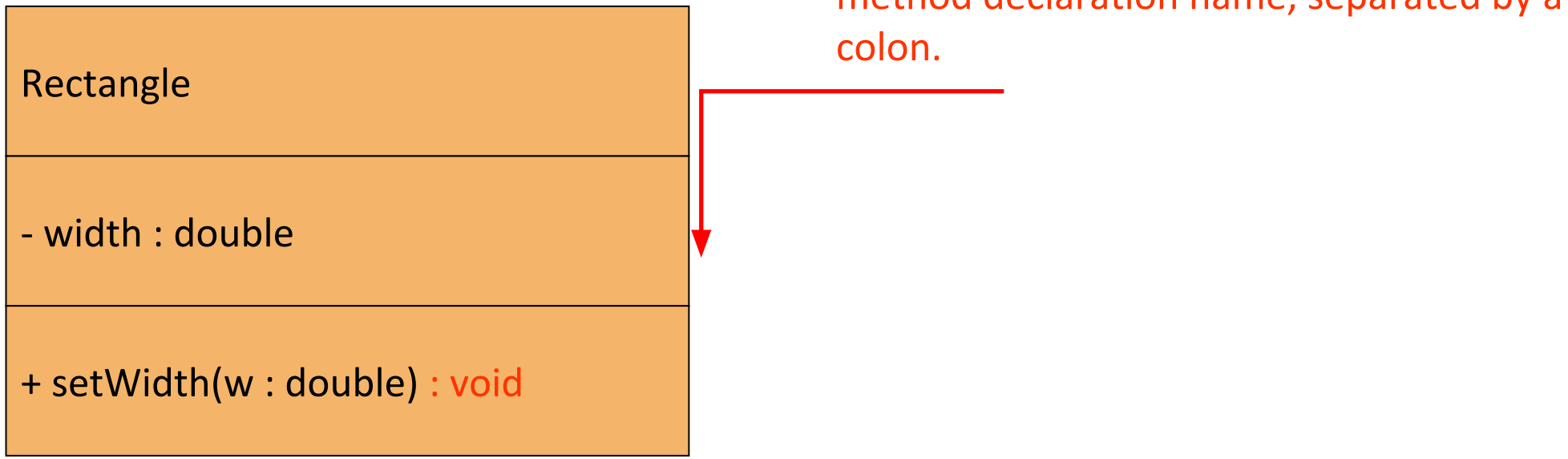
UML Data Type and Parameter Notation

- UML diagrams are language independent.
- UML diagrams use an independent notation to show return types, access modifiers, etc.



UML Data Type and Parameter Notation

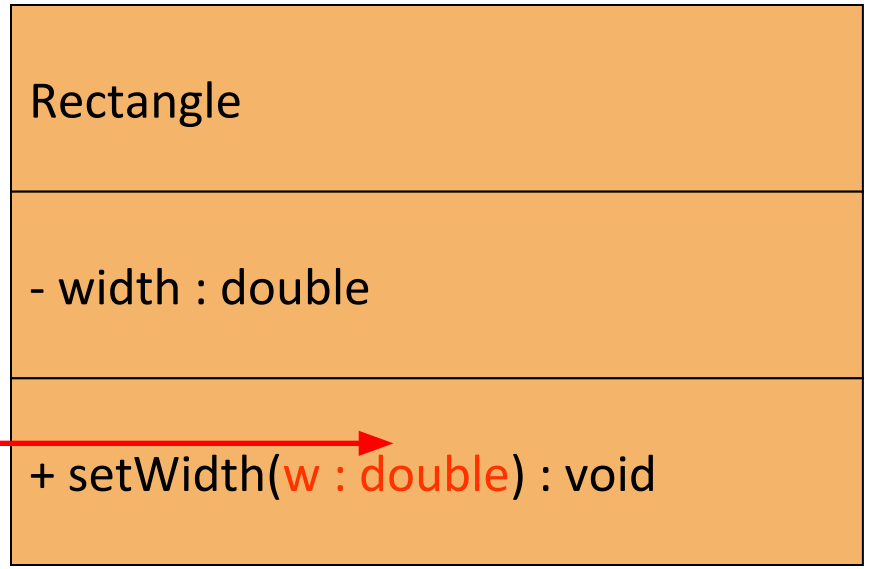
- UML diagrams are language independent.
- UML diagrams use an independent notation to show return types, access modifiers, etc.



UML Data Type and Parameter Notation

- UML diagrams are language independent.
- UML diagrams use an independent notation to show return types, access modifiers, etc.

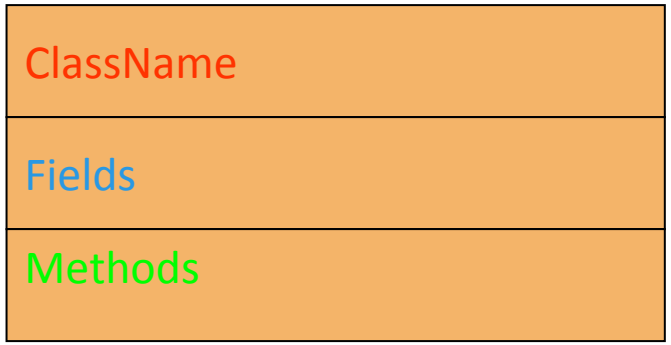
Method parameters are shown inside the parentheses using the same notation as variables.



Converting the UML Diagram to Code

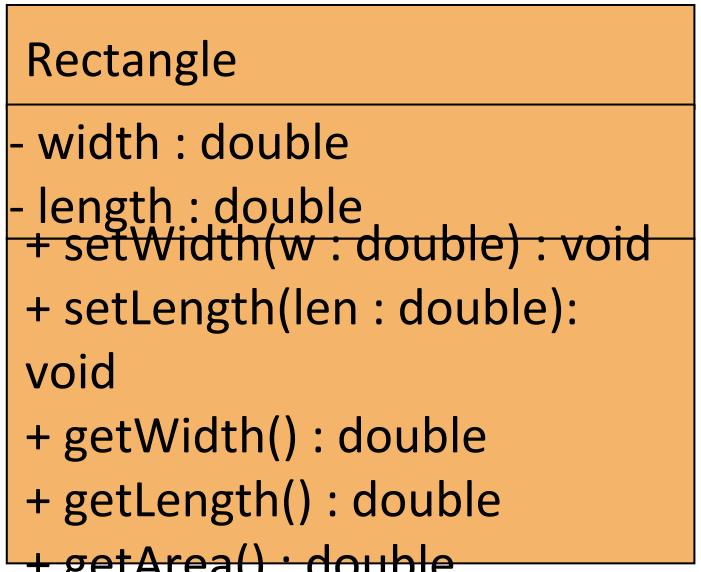
- Putting all of this information together, a Java class file can be built easily using the UML diagram.
- The UML diagram parts match the Java class file structure.

```
class header
{
        Fields
        Methods
}
```



Converting the Class Diagram to Code

The structure of the class can be compiled and tested without having bodies for the methods. Just be sure to put in dummy return values for methods that have a return type other than void.

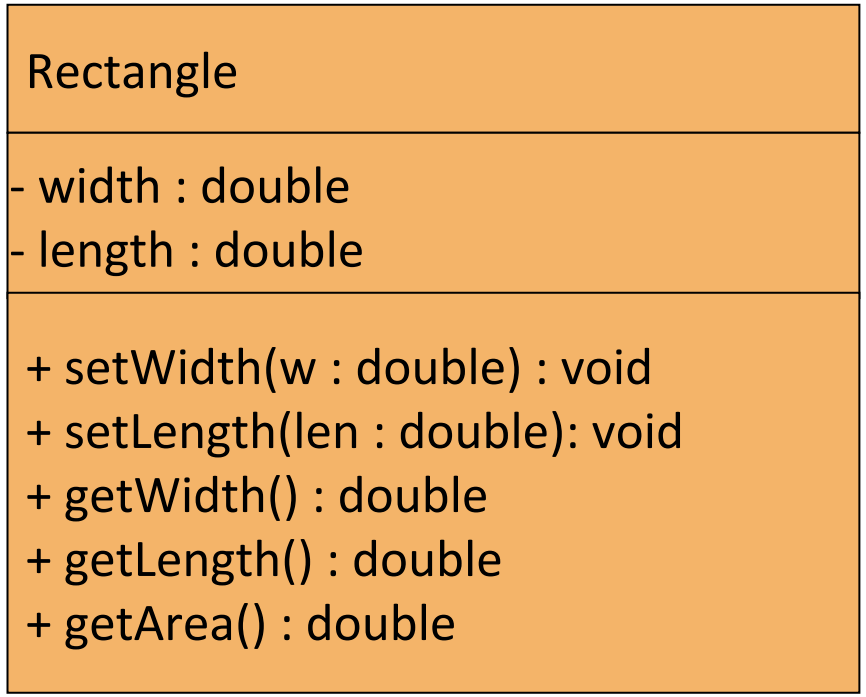



```
public class Rectangle
{
   private double width;
   private double length;

   public void setWidth(double w){
   }
   public void setLength(double len)
   {
   }
   public double getWidth(){
      return 0.0;
   }
   public double getLength()   {
      return 0.0;
   }
   public double getArea()   {
      return 0.0;
   }
```

Converting the Class Diagram to Code

Once the class structure has been tested, the method bodies can be written and tested.




```
public class Rectangle
{
   private double width;
   private double length;

   public void setWidth(double w)
{  width = w;   }
   public void setLength(double len)
   {  length = len;
   }
   public double getWidth()  {
return width;   }
   public double getLength()   {
return length; }
   public double getArea()   {
   return length * width;
   }
}
```